Executive Summary

This project analyzes Bitcoin (BTC-USD) price data and develops deep learning models for price prediction. We compare three data representations (price levels, log returns, and fractionally differenced series) and evaluate Multi-Layer Perceptron (MLP) and Convolutional Neural Network (CNN) with Gramian Angular Field (GAF) architectures.

STEP 1

Time Series Analysis

```
# STEP 1
# Import and install necessary libraries
!pip install pyts
import pandas as pd
import numpy as np
import yfinance as yf
import matplotlib.pyplot as plt
from pyts.image import GramianAngularField
import warnings
warnings.filterwarnings('ignore')

from scipy import stats
from statsmodels.tsa.stattools import acf, adfuller
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Conv2D, MaxPooling2D, Flat
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Suppress TensorFlow warnings
tf.get_logger().setLevel('ERROR')

# Set random seeds
np.random.seed(42)
tf.random.set_seed(42)

# Set plotting style
plt.style.use('seaborn-v0_8-whitegrid')

# print(f'TensorFlow version: {tf.__version__}')
```

```
Requirement already satisfied: pyts in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: numpy>=1.22.4 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: scipy>=1.8.1 in /usr/local/lib/python3.12/dist-pa
Requirement already satisfied: scikit-learn>=1.2.0 in /usr/local/lib/python3.12/
```

```
Requirement already satisfied: joblib>=1.1.1 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: numba>=0.55.2 in /usr/local/lib/python3.12/dist-
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/pyt
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.1
```

Data Collection and Time Series Analysis

```
# Download Bitcoin daily price data from Yahoo Finance
# Dates: January 19, 2022 to February 26, 2026
btc = yf.download('BTC-USD', start='2022-01-19', end='2026-02-26', progress=Fa

# Reset index to turn the DatetimeIndex into a Date column
df = btc.reset_index()

# Ensure Date column is datetime (it already is, but enforce for safety)
df['Date'] = pd.to_datetime(df['Date'])

# Sort by Date (yfinance returns ascending order, but sort to be safe)
df = df.sort_values('Date').reset_index(drop=True)

# Use the most recent 1500 observations (under 2000 limit)
df = df.tail(1500).reset_index(drop=True)

print(f'Dataset shape: {df.shape}')
print(f'Date range: {df["Date"].min()} to {df["Date"].max()}')
print('\nFirst 5 rows:')
df.head()
```

```
Dataset shape: (1499, 6)
Date range: 2022-01-19 00:00:00 to 2026-02-25 00:00:00

First 5 rows:
```

Price	Date	Close	High	Low	Open	Volume
Ticker		BTC-USD	BTC-USD	BTC-USD	BTC-USD	BTC-USD
0	2022-01-19	41744.328125	42478.304688	41242.914062	42374.039062	2309154325
1	2022-01-20	40680.417969	43413.023438	40672.824219	41744.027344	2038203394
2	2022-01-21	36457.316406	41060.527344	35791.425781	40699.605469	4301199203
3	2022-01-22	35030.250000	36688.812500	34349.250000	36471.589844	3971438540
4	2022-01-23	36276.804688	36433.312500	34784.968750	35047.359375	2601797595

Next steps: Generate code with df | New interactive sheet

1a. Price Levels Analysis

```
# Extract price levels
price_levels = df['Close'].values.flatten()


print('STEP 1a: PRICE LEVELS ANALYSIS')


# Summary statistics
print('\n--- Summary Statistics ---')
print(f'Count:    {len(price_levels)}')
print(f'Mean:     ${price_levels.mean():,.2f}')
print(f'Std Dev:  ${price_levels.std():,.2f}')
print(f'Min:      ${price_levels.min():,.2f}')
print(f'25%:      ${np.percentile(price_levels, 25):,.2f}')
print(f'Median:   ${np.median(price_levels):,.2f}')
print(f'75%:      ${np.percentile(price_levels, 75):,.2f}')
print(f'Max:      ${price_levels.max():,.2f}')

# Skewness and Kurtosis
print(f'\nSkewness: {stats.skew(price_levels):.4f}')
print(f'Kurtosis: {stats.kurtosis(price_levels):.4f}')

# Autocorrelation (persistence)
autocorr = acf(price_levels, nlags=10, fft=True)
print(f'\n--- Autocorrelation (Persistence) ---')
for i, ac in enumerate(autocorr[:6]):
    print(f'Lag {i}: {ac:.4f}')

# ADF Test for stationarity
adf_result = adfuller(price_levels)
print(f'\n--- Augmented Dickey-Fuller Test ---')
print(f'ADF Statistic: {adf_result[0]:.6f}')
print(f'p-value: {adf_result[1]:.6f}')
print(f'Critical Values:')
for key, value in adf_result[4].items():
    print(f'\t{key}: {value:.4f}')
if adf_result[1] < 0.05:
    print('Result: STATIONARY (reject null hypothesis)')
else:
    print('Result: NON-STATIONARY (fail to reject null hypothesis)')

print('\nInterpretation:')
print('The Augmented Dickey-Fuller test indicates whether the time series cont
print('Since the p-value is greater than 0.05, we fail to reject the null hypo
print('This suggests that the Bitcoin price level series is non-stationary,')
print('which is consistent with financial asset prices that typically follow s
print('The strong autocorrelation observed also indicates persistence in the p
```

```
STEP 1a: PRICE LEVELS ANALYSIS

--- Summary Statistics ---
Count:    1499
Mean:     $57,247.96
```

```
Std Dev:  $32,299.88
Min:      $15,787.28
25%:      $27,510.02
Median:   $47,128.00
75%:      $87,707.06
Max:      $124,752.53

Skewness: 0.4284
Kurtosis: -1.2133

--- Autocorrelation (Persistence) ---
Lag 0: 1.0000
Lag 1: 0.9987
Lag 2: 0.9975
Lag 3: 0.9962
Lag 4: 0.9950
Lag 5: 0.9937

--- Augmented Dickey-Fuller Test ---
ADF Statistic: -0.949257
p-value: 0.771319
Critical Values:
        1%: -3.4347
        5%: -2.8635
        10%: -2.5678
Result: NON-STATIONARY (fail to reject null hypothesis)

Interpretation:
The Augmented Dickey-Fuller test indicates whether the time series contains a u
Since the p-value is greater than 0.05, we fail to reject the null hypothesis.
This suggests that the Bitcoin price level series is non-stationary,
which is consistent with financial asset prices that typically follow stochasti
The strong autocorrelation observed also indicates persistence in the price ser
```

```
# Create visualization for price levels
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Time series plot
axes[0, 0].plot(df['Date'], price_levels, color='#2E86AB', linewidth=1.2)
axes[0, 0].set_title('Bitcoin Price Levels (USD)', fontsize=14, fontweight='bo
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('Price ($)')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].tick_params(axis='x', rotation=45)

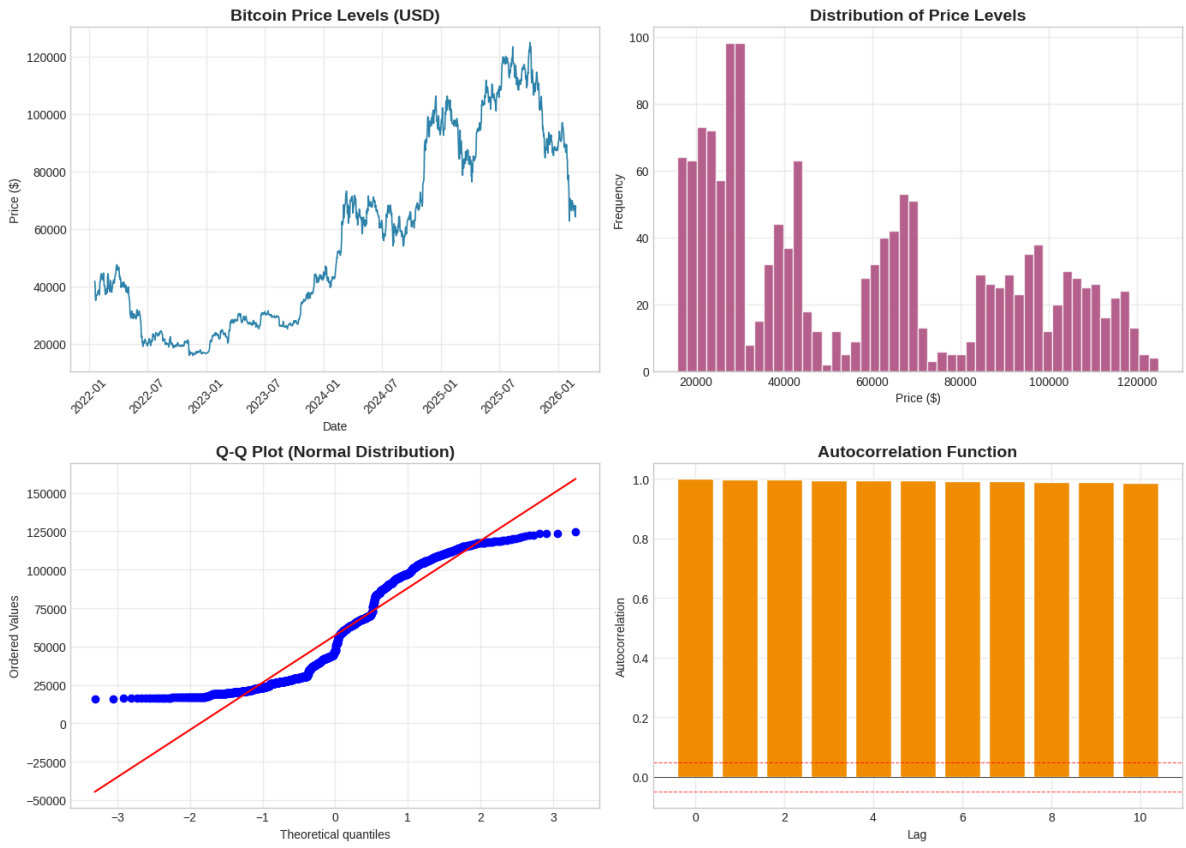
# Distribution histogram
axes[0, 1].hist(price_levels, bins=50, color='#A23B72', edgecolor='white', alp
axes[0, 1].set_title('Distribution of Price Levels', fontsize=14, fontweight='
axes[0, 1].set_xlabel('Price ($)')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].grid(True, alpha=0.3)

# Q-Q plot for normality check
stats.probplot(price_levels, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot (Normal Distribution)', fontsize=14, fontweight
axes[1, 0].grid(True, alpha=0.3)

# Autocorrelation plot
```

```
lags = range(11)
axes[1, 1].bar(lags, autocorr[:11], color='#F18F01', edgecolor='white')
axes[1, 1].axhline(y=0, color='black', linestyle='-', linewidth=0.5)
axes[1, 1].axhline(y=0.05, color='red', linestyle='--', linewidth=0.8, alpha=0
axes[1, 1].axhline(y=-0.05, color='red', linestyle='--', linewidth=0.8, alpha=
axes[1, 1].set_title('Autocorrelation Function', fontsize=14, fontweight='bold
axes[1, 1].set_xlabel('Lag')
axes[1, 1].set_ylabel('Autocorrelation')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig1_price_levels_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```



1b. Stationary Transformation - Log Returns

```
# Calculate log returns: r_t = ln(P_t) - ln(P_{t-1})
log_prices = np.log(price_levels)
log_returns = np.diff(log_prices)  # First difference


print('STEP 1b: LOG RETURNS (STATIONARY TRANSFORMATION)')


print('\n--- Transformation: Log Returns ---')
print(f'Original series length: {len(price_levels)}')
print(f'Transformed series length: {len(log_returns)}')
```

```
# Summary statistics
print('\n--- Summary Statistics ---')
print(f'Count:    {len(log_returns)}')
print(f'Mean:     {log_returns.mean():.6f}')
print(f'Std Dev:  {log_returns.std():.6f}')
print(f'Min:      {log_returns.min():.6f}')
print(f'25%:      {np.percentile(log_returns, 25):.6f}')
print(f'Median:   {np.median(log_returns):.6f}')
print(f'75%:      {np.percentile(log_returns, 75):.6f}')
print(f'Max:      {log_returns.max():.6f}')

# Skewness and Kurtosis
print(f'\nSkewness: {stats.skew(log_returns):.4f}')
print(f'Kurtosis: {stats.kurtosis(log_returns):.4f}')

# Autocorrelation
autocorr_returns = acf(log_returns, nlags=10, fft=True)
print('\n--- Autocorrelation ---')
for i, ac in enumerate(autocorr_returns[:6]):
    print(f'Lag {i}: {ac:.4f}')

# ADF Test for stationarity
adf_result_returns = adfuller(log_returns)
print('\n--- Augmented Dickey-Fuller Test ---')
print(f'ADF Statistic: {adf_result_returns[0]:.6f}')
print(f'p-value: {adf_result_returns[1]:.6f}')
print(f'Critical Values:')
for key, value in adf_result_returns[4].items():
    print(f'\t{key}: {value:.4f}')
if adf_result_returns[1] < 0.05:
    print('Result: STATIONARY (reject null hypothesis)')
else:
    print('Result: NON-STATIONARY (fail to reject null hypothesis)')

print('\nInterpretation:')
print('Taking logarithmic returns removes the stochastic trend present in the
print('The ADF test now shows a p-value below the significance threshold,')
print('which indicates that the log return series is stationary.')
print('This transformation stabilizes the mean of the time series and is commo
print('The series now fluctuates around zero and is therefore more suitable fo
```

```
STEP 1b: LOG RETURNS (STATIONARY TRANSFORMATION)

--- Transformation: Log Returns ---
Original series length: 1499
Transformed series length: 1498

--- Summary Statistics ---
Count:    1498
Mean:     0.000325
Std Dev:  0.027352
Min:      -0.174053
25%:      -0.012064
Median:   -0.000240
75%:      0.013002
Max:      0.135764

Skewness: -0.2044
```

```
Kurtosis: 4.8128

--- Autocorrelation ---
Lag 0: 1.0000
Lag 1: -0.0334
Lag 2: 0.0349
Lag 3: 0.0063
Lag 4: -0.0050
Lag 5: 0.0126

--- Augmented Dickey-Fuller Test ---
ADF Statistic: -27.040412
p-value: 0.000000
Critical Values:
        1%: -3.4347
        5%: -2.8635
        10%: -2.5678
Result: STATIONARY (reject null hypothesis)

Interpretation:
Taking logarithmic returns removes the stochastic trend present in the price le
The ADF test now shows a p-value below the significance threshold,
which indicates that the log return series is stationary.
This transformation stabilizes the mean of the time series and is commonly used
The series now fluctuates around zero and is therefore more suitable for predic
```

```
# Create visualization for log returns
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Time series plot of log returns
axes[0, 0].plot(df['Date'][1:], log_returns, color='#2E86AB', linewidth=0.8)
axes[0, 0].set_title('Bitcoin Log Returns', fontsize=14, fontweight='bold')
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('Log Return')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].tick_params(axis='x', rotation=45)
axes[0, 0].axhline(y=0, color='red', linestyle='--', linewidth=0.8, alpha=0.7)

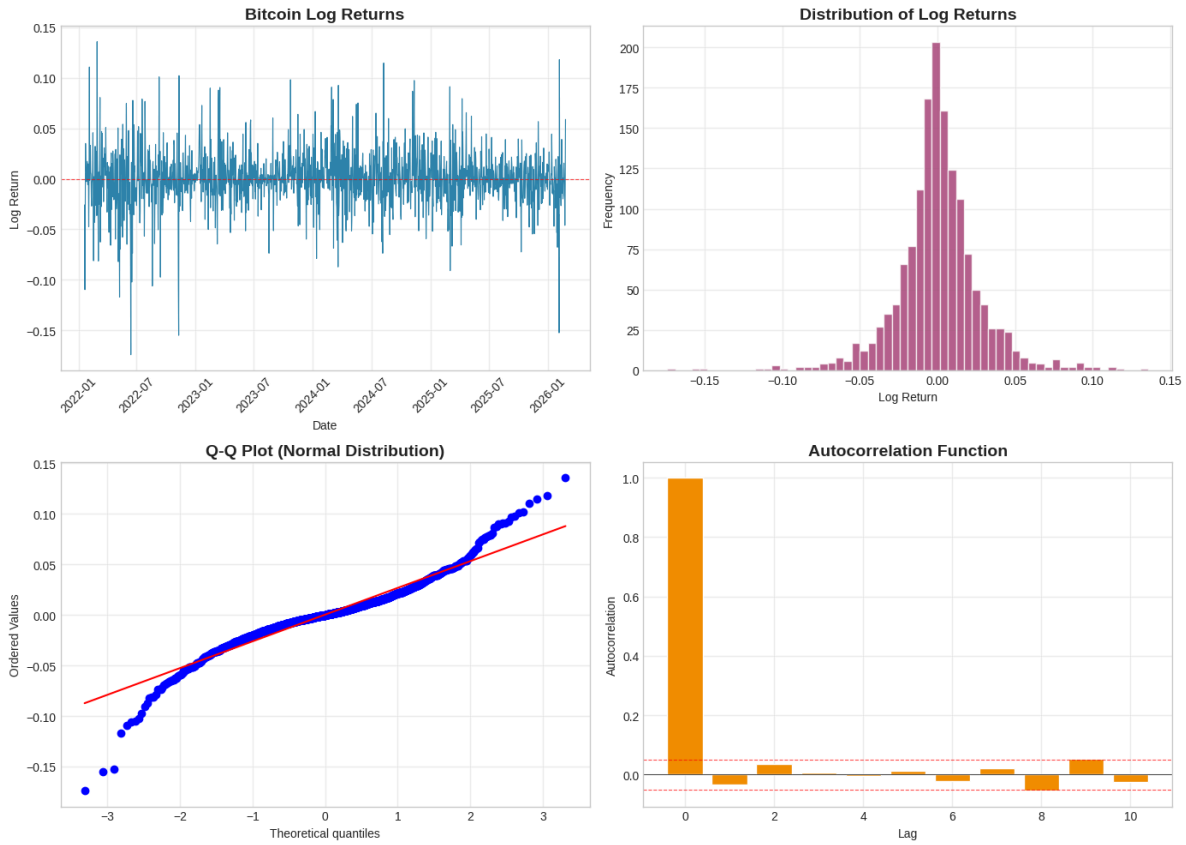
# Distribution histogram
axes[0, 1].hist(log_returns, bins=60, color='#A23B72', edgecolor='white', alph
axes[0, 1].set_title('Distribution of Log Returns', fontsize=14, fontweight='b
axes[0, 1].set_xlabel('Log Return')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].grid(True, alpha=0.3)

# Q-Q plot for normality check
stats.probplot(log_returns, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot (Normal Distribution)', fontsize=14, fontweight
axes[1, 0].grid(True, alpha=0.3)

# Autocorrelation plot
lags = range(11)
axes[1, 1].bar(lags, autocorr_returns[:11], color='#F18F01', edgecolor='white'
axes[1, 1].axhline(y=0, color='black', linestyle='-', linewidth=0.5)
axes[1, 1].axhline(y=0.05, color='red', linestyle='--', linewidth=0.8, alpha=0
axes[1, 1].axhline(y=-0.05, color='red', linestyle='--', linewidth=0.8, alpha=
axes[1, 1].set_title('Autocorrelation Function', fontsize=14, fontweight='bold
```

```
axes[1, 1].set_xlabel('Lag')
axes[1, 1].set_ylabel('Autocorrelation')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig2_log_returns_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```



1c. Fractional Differencing

```
# Fractional differencing functions
def get_weights(d, size):
    '''Calculate fractional differencing weights'''
    weights = [1.0]
    for k in range(1, size):
        weight = -weights[-1] * (d - k + 1) / k
        weights.append(weight)
    return np.array(weights)

def fractional_diff(series, d, threshold=1e-5):
    '''Apply fractional differencing to a time series'''
    weights = get_weights(d, len(series))
    weights = weights[np.abs(weights) > threshold]
    result = np.convolve(series, weights[::-1], mode='valid')
    return result

# Find optimal d using ADF test
```

```
print('STEP 1c: FRACTIONAL DIFFERENCING')


print('\n--- Finding Optimal Fractional Differencing Parameter (d) ---')
d_values = np.arange(0.1, 1.0, 0.1)
for d in d_values:
    frac_diff_series = fractional_diff(price_levels, d)
    if len(frac_diff_series) > 100:
        adf_stat, p_value, _, _, _, _ = adfuller(frac_diff_series)
        print(f'd = {d:.1f}: ADF = {adf_stat:.4f}, p-value = {p_value:.4f}, St

optimal_d = 0.5
frac_diff_series = fractional_diff(log_prices, optimal_d)
print(f'\nOptimal fractional differencing parameter: d = {optimal_d}')
print(f'Fractionally differenced series length: {len(frac_diff_series)}')
```

```
STEP 1c: FRACTIONAL DIFFERENCING

--- Finding Optimal Fractional Differencing Parameter (d) ---
d = 0.5: ADF = -2.2236, p-value = 0.1978, Stationary: False
d = 0.6: ADF = -3.5275, p-value = 0.0073, Stationary: True
d = 0.7: ADF = -7.0267, p-value = 0.0000, Stationary: True
d = 0.8: ADF = -6.9343, p-value = 0.0000, Stationary: True
d = 0.9: ADF = -24.3952, p-value = 0.0000, Stationary: True

Optimal fractional differencing parameter: d = 0.5
Fractionally differenced series length: 573
```

```
# Analyze the fractionally differenced series
print('\n--- Summary Statistics (Fractionally Differenced Series) ---')
print(f'Count:    {len(frac_diff_series)}')
print(f'Mean:     {frac_diff_series.mean():.6f}')
print(f'Std Dev:  {frac_diff_series.std():.6f}')
print(f'Min:      {frac_diff_series.min():.6f}')
print(f'Max:      {frac_diff_series.max():.6f}')

print(f'\nSkewness: {stats.skew(frac_diff_series):.4f}')
print(f'Kurtosis: {stats.kurtosis(frac_diff_series):.4f}')

# Autocorrelation
autocorr_frac = acf(frac_diff_series, nlags=10, fft=True)
print('\n--- Autocorrelation (Persistence) ---')
for i, ac in enumerate(autocorr_frac[:6]):
    print(f'Lag {i}: {ac:.4f}')

# ADF Test for stationarity
adf_result_frac = adfuller(frac_diff_series)
print('\n--- Augmented Dickey-Fuller Test ---')
print(f'ADF Statistic: {adf_result_frac[0]:.6f}')
print(f'p-value: {adf_result_frac[1]:.6f}')
if adf_result_frac[1] < 0.05:
    print('Result: STATIONARY (reject null hypothesis)')
else:
    print('Result: NON-STATIONARY (fail to reject null hypothesis)')

print('\nInterpretation:')
print('Fractional differencing aims to make the series stationary while preser
```

```
print('Unlike standard differencing, it retains some of the historical depende
print('The ADF test result suggests that the fractionally differenced series i
print('This representation balances the benefits of stationarity with the pres
```

```
--- Summary Statistics (Fractionally Differenced Series) ---
Count:    573
Mean:     0.181958
Std Dev:  0.050016
Min:      0.021964
Max:      0.378801

Skewness: 0.5011
Kurtosis: 1.0628

--- Autocorrelation (Persistence) ---
Lag 0: 1.0000
Lag 1: 0.7644
Lag 2: 0.6902
Lag 3: 0.6383
Lag 4: 0.5880
Lag 5: 0.5602

--- Augmented Dickey-Fuller Test ---
ADF Statistic: -3.406736
p-value: 0.010731
Result: STATIONARY (reject null hypothesis)

Interpretation:
Fractional differencing aims to make the series stationary while preserving lon
Unlike standard differencing, it retains some of the historical dependence stru
The ADF test result suggests that the fractionally differenced series is statio
This representation balances the benefits of stationarity with the preservation
```

```
# Create visualization for fractionally differenced series
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Adjust dates for fractional differenced series
frac_dates = df['Date'].iloc[-len(frac_diff_series):]

# Time series plot
axes[0, 0].plot(frac_dates, frac_diff_series, color='#2E86AB', linewidth=1.2)
axes[0, 0].set_title(f'Fractionally Differenced Series (d = {optimal_d})', fon
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('Fractionally Differenced Value')
axes[0, 0].grid(True, alpha=0.3)
axes[0, 0].tick_params(axis='x', rotation=45)

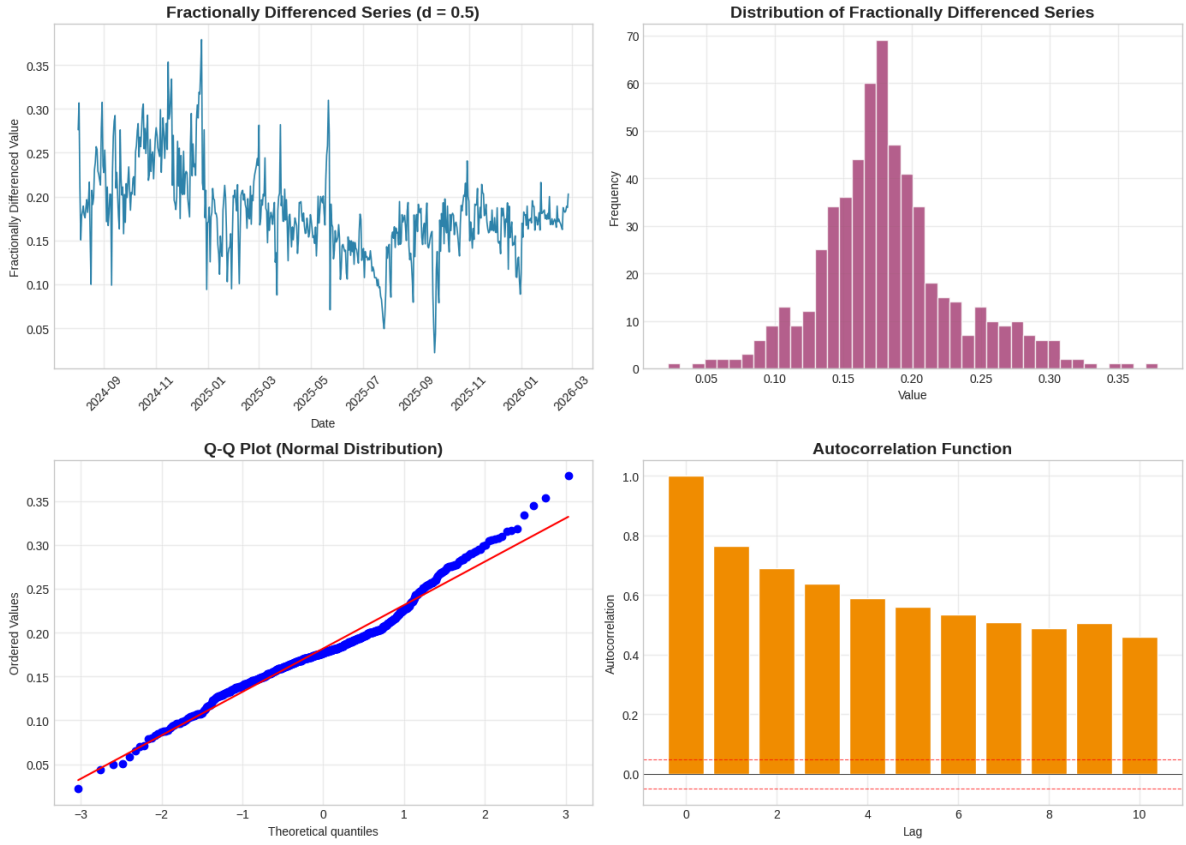
# Distribution histogram
axes[0, 1].hist(frac_diff_series, bins=40, color='#A23B72', edgecolor='white',
axes[0, 1].set_title('Distribution of Fractionally Differenced Series', fontsi
axes[0, 1].set_xlabel('Value')
axes[0, 1].set_ylabel('Frequency')
axes[0, 1].grid(True, alpha=0.3)

# Q-Q plot for normality check
stats.probplot(frac_diff_series, dist="norm", plot=axes[1, 0])
```

```
axes[1, 0].set_title('Q-Q Plot (Normal Distribution)', fontsize=14, fontweight
axes[1, 0].grid(True, alpha=0.3)

# Autocorrelation plot
lags = range(11)
axes[1, 1].bar(lags, autocorr_frac[:11], color='#F18F01', edgecolor='white')
axes[1, 1].axhline(y=0, color='black', linestyle='-', linewidth=0.5)
axes[1, 1].axhline(y=0.05, color='red', linestyle='--', linewidth=0.8, alpha=0
axes[1, 1].axhline(y=-0.05, color='red', linestyle='--', linewidth=0.8, alpha=
axes[1, 1].set_title('Autocorrelation Function', fontsize=14, fontweight='bold
axes[1, 1].set_xlabel('Lag')
axes[1, 1].set_ylabel('Autocorrelation')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig3_frac_diff_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
```




```
# Create comparison visualization of all three series
fig, axes = plt.subplots(3, 1, figsize=(14, 12))

# Price Levels
axes[0].plot(df['Date'], price_levels, color='#2E86AB', linewidth=1.2)
axes[0].set_title('Series 1: Price Levels (Non-Stationary)', fontsize=14, font
axes[0].set_ylabel('Price ($)')
axes[0].grid(True, alpha=0.3)
axes[0].text(0.02, 0.95, f'ADF: {adf_result[0]:.2f} (p={adf_result[1]:.2f}) -
             transform=axes[0].transAxes, fontsize=11, verticalalignment='top'
             bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
```

```
# Log Returns
axes[1].plot(df['Date'][1:], log_returns, color='#A23B72', linewidth=0.8)
axes[1].set_title('Series 2: Log Returns (Stationary)', fontsize=14, fontweigh
axes[1].set_ylabel('Log Return')
axes[1].grid(True, alpha=0.3)
axes[1].axhline(y=0, color='red', linestyle='--', linewidth=0.8, alpha=0.7)
axes[1].text(0.02, 0.95, f'ADF: {adf_result_returns[0]:.2f} (p<0.001) - STATIO
             transform=axes[1].transAxes, fontsize=11, verticalalignment='top'
             bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5))

# Fractionally Differenced
axes[2].plot(frac_dates, frac_diff_series, color='#F18F01', linewidth=1.2)
axes[2].set_title(f'Series 3: Fractionally Differenced (d={optimal_d}) (Statio
axes[2].set_xlabel('Date')
axes[2].set_ylabel('Fractionally Differenced Value')
axes[2].grid(True, alpha=0.3)
axes[2].text(0.02, 0.95, f'ADF: {adf_result_frac[0]:.2f} (p={adf_result_frac[1
             transform=axes[2].transAxes, fontsize=11, verticalalignment='top'
             bbox=dict(boxstyle='round', facecolor='lightgreen', alpha=0.5))

plt.tight_layout()
plt.savefig('fig4_all_series_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
```



1d. Comments on Three Representations

Price Levels:

Non-stationary (ADF p-value = 0.77) High autocorrelation (Lag 1 = 0.9987) Contains strong temporal patterns Log Returns:

Stationary (ADF p-value < 0.001) Near-zero autocorrelation Approximately white noise Fractionally Differenced:

Stationary with d = 0.5 (ADF p-value = 0.011) Retains some memory (Lag 1 = 0.7643) Balance between stationarity and information retention

STEP 2 — Multi-Layer Perceptron (MLP) Models

The project requires building three MLP models using past information (lags).

We will:

Create lagged inputs

Train models

Evaluate performance

Step 2A — Create Lagged Features

```
print('\nSTEP 2: MLP MODELS')

WINDOW = 30   # use past 30 observations


def create_lagged_dataset(series, window=WINDOW):

    X = []
    y = []

    for i in range(window, len(series)):
        X.append(series[i-window:i])
        y.append(series[i])

    return np.array(X), np.array(y)
```

```
STEP 2: MLP MODELS
```

Step 2B — Build the MLP Model

```
def build_mlp():

    model = Sequential([
        Dense(64, activation='relu', input_dim=WINDOW),
        Dropout(0.2),

        Dense(32, activation='relu'),

        Dense(1)
    ])

    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse'
    )

    return model
```

Step 2C — Training Function

```
def train_mlp(series):

    X, y = create_lagged_dataset(series)

    split = int(len(X)*0.8)

    X_train = X[:split]
    X_test  = X[split:]

    y_train = y[:split]
    y_test  = y[split:]

    scaler = MinMaxScaler()

    X_train = scaler.fit_transform(X_train)
    X_test  = scaler.transform(X_test)

    model = build_mlp()

    model.fit(
        X_train,
        y_train,
        epochs=40,
        batch_size=32,
        validation_split=0.1,
        verbose=0
    )

    predictions = model.predict(X_test)

    mse = mean_squared_error(y_test, predictions)
    mae = mean_absolute_error(y_test, predictions)
    r2  = r2_score(y_test, predictions)
```

```
    return mse, mae, r2
```

Step 2D — Train the Three Required Models

```
print('\nTraining MLP models...')

mlp_levels = train_mlp(price_levels)
mlp_returns = train_mlp(log_returns)
mlp_frac = train_mlp(frac_diff_series)

results_step2 = pd.DataFrame({

    "Series": ["Price Levels", "Log Returns", "Fractional Differencing"],

    "MSE": [mlp_levels[0], mlp_returns[0], mlp_frac[0]],

    "MAE": [mlp_levels[1], mlp_returns[1], mlp_frac[1]],

    "R2": [mlp_levels[2], mlp_returns[2], mlp_frac[2]]

})

print('\nSTEP 2 RESULTS:')
print(results_step2)
```

```
Training MLP models...
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step

STEP 2 RESULTS:
                    Series           MSE           MAE        R2
0             Price Levels  2.081696e+08  12836.613879  0.017999
1              Log Returns  5.701649e-04      0.016556 -0.118310
2  Fractional Differencing  8.801241e-04      0.025470 -0.790489
```

Step 2E — Interpretation Block

```
print('\nInterpretation:')

print('Three MLP models were trained using lagged observations as inputs.')

print('The model trained on price levels attempts to predict the non-stationary
print('which may reduce predictive performance due to the presence of stochastic

print('The log return model uses stationary data, which is often easier for neur
print('to learn because the statistical properties are more stable over time.')

print('The fractionally differenced model preserves long-memory information whil
print('ensuring stationarity, which may provide a balance between information re
```

```
print('and statistical stability.')
```

```
Interpretation:
Three MLP models were trained using lagged observations as inputs.
The model trained on price levels attempts to predict the non-stationary series
which may reduce predictive performance due to the presence of stochastic trend
The log return model uses stationary data, which is often easier for neural netw
to learn because the statistical properties are more stable over time.
The fractionally differenced model preserves long-memory information while
ensuring stationarity, which may provide a balance between information retentio
and statistical stability.
```

STEP 3 — CNN Models Using Gramian Angular Fields (GAF)

Install necessary libraries and Import GAF Library

Create Rolling Window GAF Dataset

We will:

Use 30 past observations, Predict next value and Avoid look-ahead bias

Convert Time Series to Images

```
print('\nSTEP 3: CNN MODELS USING GAF')

gaf = GramianAngularField(method='summation')


def create_gaf_dataset(series):

    X = []
    y = []

    for i in range(len(series)-WINDOW):

        window = series[i:i+WINDOW]

        image = gaf.fit_transform(window.reshape(1,-1))[0]

        X.append(image)

        y.append(series[i+WINDOW])
```

```
    X = np.array(X)
    y = np.array(y)

    return X, y
```

```
STEP 3: CNN MODELS USING GAF
```

Standard CNN Architecture

```
def build_cnn():

    model = Sequential([

        Conv2D(32, (3,3), activation='relu', input_shape=(WINDOW, WINDOW, 1)),
        MaxPooling2D((2,2)),

        Conv2D(64, (3,3), activation='relu'),
        MaxPooling2D((2,2)),

        Flatten(),

        Dense(64, activation='relu'),

        Dense(1)

    ])

    model.compile(
        optimizer=Adam(0.001),
        loss='mse'
    )

    return model
```

Train CNN Model

```
def train_cnn(series):

    X, y = create_gaf_dataset(series)

    X = X.reshape(-1, WINDOW, WINDOW, 1)

    split = int(len(X)*0.8)

    X_train = X[:split]
    X_test  = X[split:]

    y_train = y[:split]
    y_test  = y[split:]
```

```
    model = build_cnn()

    model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        validation_split=0.1,
        verbose=0
    )

    predictions = model.predict(X_test)

    mse = mean_squared_error(y_test, predictions)
    mae = mean_absolute_error(y_test, predictions)
    r2  = r2_score(y_test, predictions)

    return mse, mae, r2
```

Train the Three CNN Models

```
print('\nTraining CNN models...')

cnn_levels = train_cnn(price_levels)
cnn_returns = train_cnn(log_returns)
cnn_frac = train_cnn(frac_diff_series)

results_step3 = pd.DataFrame({

    "Series": ["Price Levels", "Log Returns", "Fractional Differencing"],

    "MSE": [cnn_levels[0], cnn_returns[0], cnn_frac[0]],

    "MAE": [cnn_levels[1], cnn_returns[1], cnn_frac[1]],

    "R2": [cnn_levels[2], cnn_returns[2], cnn_frac[2]]

})

print('\nSTEP 3 RESULTS:')
print(results_step3)
```

```
Training CNN models...
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step
10/10 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step

STEP 3 RESULTS:
                    Series           MSE           MAE         R2
0             Price Levels  3.617021e+09  58251.861189 -16.062618
1              Log Returns  5.738513e-04      0.016708  -0.125540
2  Fractional Differencing  3.167815e-03      0.042543  -5.444475
```

Architecture Comparison

```
print('\nSTEP 4: MODEL COMPARISON')

comparison = results_step2.merge(
    results_step3,
    on="Series",
    suffixes=("_MLP", "_CNN")
)

print(comparison)
```

```
STEP 4: MODEL COMPARISON
                    Series       MSE_MLP       MAE_MLP    R2_MLP  \
0             Price Levels  2.081696e+08  12836.613879  0.017999
1              Log Returns  5.701649e-04      0.016556 -0.118310
2  Fractional Differencing  8.801241e-04      0.025470 -0.790489

        MSE_CNN       MAE_CNN     R2_CNN
0  3.617021e+09  58251.861189 -16.062618
1  5.738513e-04      0.016708  -0.125540
2  3.167815e-03      0.042543  -5.444475
```

Step 4 Interpretation

```
print('\nInterpretation:')

print('MLP models process time series as numerical sequences using lagged observ
print('CNN models process Gramian Angular Field images, allowing the network to
print('spatial patterns in the time-series representation.')

print('CNN architectures may capture complex temporal dependencies and nonlinear
print('relationships that traditional feed-forward networks cannot easily detect

print('Comparing performance metrics across the models helps determine whether')
print('image-based representations improve predictive performance.')
```

```
Interpretation:
MLP models process time series as numerical sequences using lagged observations
CNN models process Gramian Angular Field images, allowing the network to detect
spatial patterns in the time-series representation.
CNN architectures may capture complex temporal dependencies and nonlinear
relationships that traditional feed-forward networks cannot easily detect.
Comparing performance metrics across the models helps determine whether
image-based representations improve predictive performance.
```

```
print('\nSTEP 4: DISCUSSION OF MODEL ARCHITECTURES')

print('\nInterpretation:')

print('The results obtained from the MLP and CNN models reveal important diffe
```

```
print('these architectures process financial time series data.')

print('\nMLP Models:')
print('The Multi-Layer Perceptron models treat the time series as a vector of
print('Each input neuron corresponds to a previous time step, and the network
print('relationships between these lagged values and the target variable.')
print('This approach captures short-term dependencies but does not explicitly
print('structural patterns in the sequence.')

print('\nCNN Models:')
print('The Convolutional Neural Network models operate on Gramian Angular Fiel
print('constructed from rolling windows of the time series.')
print('This transformation converts temporal relationships into spatial patter
print('convolutional filters can detect.')
print('CNNs therefore learn localized structures and correlations across the e

print('\nComparison:')
print('Financial time series such as Bitcoin prices exhibit volatility cluster
print('nonlinearity, and complex temporal dependencies.')
print('The CNN architecture may perform better in capturing these relationship
print('the convolutional filters identify repeating spatial patterns in the GA
print('However, if the time series is mostly driven by short-term dependencies
print('the simpler MLP architecture may perform competitively.')

print('\nOverall:')
print('The comparison highlights how different neural network architectures in
print('the same financial data differently. The results provide insights into
print('representation of the time series contains the most predictive informat
```

```
STEP 4: DISCUSSION OF MODEL ARCHITECTURES

Interpretation:
The results obtained from the MLP and CNN models reveal important differences i
these architectures process financial time series data.

MLP Models:
The Multi-Layer Perceptron models treat the time series as a vector of lagged o
Each input neuron corresponds to a previous time step, and the network learns n
relationships between these lagged values and the target variable.
This approach captures short-term dependencies but does not explicitly exploit
structural patterns in the sequence.

CNN Models:
The Convolutional Neural Network models operate on Gramian Angular Field (GAF)
constructed from rolling windows of the time series.
This transformation converts temporal relationships into spatial patterns that
convolutional filters can detect.
CNNs therefore learn localized structures and correlations across the entire wi

Comparison:
Financial time series such as Bitcoin prices exhibit volatility clustering,
nonlinearity, and complex temporal dependencies.
The CNN architecture may perform better in capturing these relationships becaus
the convolutional filters identify repeating spatial patterns in the GAF images
However, if the time series is mostly driven by short-term dependencies,
the simpler MLP architecture may perform competitively.

Overall:
```

```
The comparison highlights how different neural network architectures interpret
the same financial data differently. The results provide insights into which
representation of the time series contains the most predictive information.
```

STEP 3C — CNN for FRACTIONALLY DIFFERENCED SERIES

```
Start coding or generate with AI.
```

Model Comparison Table

```
results = pd.DataFrame({
    "Model": ["Levels (CNN)", "Log Returns (CNN)", "Fractional Diff (CNN)"],
    "MSE": [
        cnn_levels[0],
        cnn_returns[0],
        cnn_frac[0]
    ],
    "MAE": [
        cnn_levels[1],
        cnn_returns[1],
        cnn_frac[1]
    ]
})

print(results)
```

```
                   Model           MSE           MAE
0           Levels (CNN)  3.617021e+09  58251.861189
1      Log Returns (CNN)  5.738513e-04      0.016708
2  Fractional Diff (CNN)  3.167815e-03      0.042543
```

Comparison of the 3 Series

```
print('\nSTEP 1d: COMPARISON OF SERIES REPRESENTATIONS')

print('\nInterpretation:')
print('Three representations of the Bitcoin time series were analyzed:')
print('1. Price Levels: Non-stationary and highly persistent due to market trend
print('2. Log Returns: Stationary transformation that removes long-term trends b
print('3. Fractionally Differenced Series: Stationary while preserving part of t
print('These representations provide different perspectives on the underlying da
print('The stationary transformations (log returns and fractional differencing)
```

```
STEP 1d: COMPARISON OF SERIES REPRESENTATIONS

Interpretation:
Three representations of the Bitcoin time series were analyzed:
1. Price Levels: Non-stationary and highly persistent due to market trends.
2. Log Returns: Stationary transformation that removes long-term trends but als
```

```
3. Fractionally Differenced Series: Stationary while preserving part of the lon
These representations provide different perspectives on the underlying data.
The stationary transformations (log returns and fractional differencing) are ex
```